Ваш первый проект в GitLab

Занятие 08 · День 4, после обеда (часть 1)

Группы, проекты, доступы, issues и merge requests

Структура GitLab

```
GitLab
└── Группа  demo                  ← команда / отдел
    ├── Подгруппа  demo/infra
    │   └── Проект  k8s-manifests
    └── Проект  demo/shop-api      ← репозиторий + issues + CI + registry
```

- **Проект** = Git-репозиторий + issues + MR + CI/CD + Container Registry + wiki
- **Группа** объединяет проекты: общие участники, переменные CI, runners
- Путь проекта = URL: `http://gitlab.192.168.56.200.nip.io/demo/shop-api`
- Видимость: **Private**, **Internal** (все вошедшие), **Public**

Роли

Роль	Может
Guest	видеть issues (в private-проекте не видит код)
Planner	управлять issues, эпиками, планированием
Reporter	читать код, скачивать образы из registry
Developer	push в незащищённые ветки, создавать MR, запускать пайплайны
Maintainer	merge в защищённые ветки, настройки CI/CD, runners, переменные
Owner	всё, включая удаление и видимость

Роль, выданная в **группе**, наследуется всеми проектами группы.

Доступ к репозиторию: HTTPS или SSH

HTTP(S) + Personal Access Token

User settings → **Access tokens** → scope `write_repository`

```
git clone http://gitlab.192.168.56.200.nip.io/demo/shop-api.git
# Username: ваш логин
# Password: glpat-... (токен!)
git config --global credential.helper store
```

SSH-ключ

```
ssh-keygen -t ed25519 -C "me@master-1"
cat ~/.ssh/id_ed25519.pub
```

User settings → **SSH Keys** → вставить публичный ключ

`git clone git@gitlab…:demo/shop-api.git`

На стенде используем **HTTP + токен** — проще и не нужен проброс порта 22. Пароль аккаунта вместо токена не работает при включённой 2FA.

ДЕМО Создаём группу и проект

1. **New group** → `demo`, Private
2. **New project → Create blank project** → `shop-api` в группе `demo`
   - снять галочку «Initialize repository with a README» — зальём существующий код
3. Отправляем код:

```
cd ~/shop-api           # код демо-приложения (пока без .gitlab-ci.yml)
git init -b main
git add . && git commit -m "feat: shop-api 1.0"
git remote add origin http://gitlab.192.168.56.200.nip.io/demo/shop-api.git
git push -u origin main
```

Что есть в проекте

Раздел	Зачем
Code → Repository	файлы, история, ветки, теги, сравнение
Plan → Issues / Boards / Milestones	задачи, канбан-доска, вехи
Code → Merge requests	предложение изменений + обсуждение + CI
Build → Pipelines / Jobs	CI/CD (появится, когда добавим `.gitlab-ci.yml` и runner)
Deploy → Container Registry	образы проекта
Operate → Environments	куда и что выкачено
Settings	участники, защита веток, CI/CD-переменные, runners, токены

Web IDE (клавиша `.` на странице репозитория) — правки прямо в браузере.

Issues: работа начинается с задачи

- Заголовок, описание (Markdown), **исполнитель**, **метки**, **milestone**, срок
- Шаблоны: `.gitlab/issue_templates/Bug.md`
- **Boards** — колонки по меткам: `To Do` → `Doing` → `Review`
- Связь с кодом:
  - `#12` в коммите/MR — ссылка на issue
  - `Closes #12` в описании MR — закроет issue при слиянии
- Кнопка **Create merge request** из issue: создаёт ветку `12-название` и черновик MR

Merge Request — сердце совместной работы

1. Ветка `feature/...` → изменения → push
2. **MR**: описание, ссылка на issue, reviewers
3. **Пайплайн** проверяет изменения (с занятия 09)
4. **Review**: комментарии к строкам, suggestions, threads → resolve
5. **Approve** → **Merge** (в CE одобрение рекомендательное; обязательные — в Premium)
6. Ветка удаляется, issue закрывается

Настройки (Settings → Merge requests): merge method (merge commit / fast-forward), **squash**, «Pipelines must succeed», «All threads must be resolved».

Защищённые ветки и теги

Settings → Repository → Protected branches

Ветка	Allowed to merge	Allowed to push
`main`	Maintainers	**No one**

- В `main` — только через MR; никаких `push --force`
- **Protected tags** `v*` — релизы создают только Maintainers
- **Protected-переменные CI** доступны только пайплайнам защищённых веток и тегов — так production-доступы не утекут в feature-ветку

Container Registry проекта

- Образы проекта: `registry.192.168.56.200.nip.io/demo/shop-api:<тег>`
- В CI доступ — автоматически: `CI_REGISTRY`, `CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`
- Снаружи — по токену:
  - **Personal Access Token** (`read_registry` / `write_registry`) — для людей
  - **Deploy token** — для серверов и кластера (только чтение образов)
- **Cleanup policies** — удаление старых тегов, иначе диск закончится

```
docker login registry.192.168.56.200.nip.io     # логин + PAT
```

Токены в GitLab — не перепутать

Токен	Префикс	Кто использует
Personal access token	`glpat-`	человек: git, API, registry
Project / Group access token	`glpat-`	«бот» проекта/группы
Deploy token	`gldt-`	сервер / кластер: clone, pull образов
Runner authentication token	`glrt-`	GitLab Runner (занятие 09)
CI job token	`CI_JOB_TOKEN`	job во время пайплайна, живёт пока идёт job
Agent token	`glagent-`	GitLab agent for Kubernetes (занятие 11)

Всегда: **минимальные scopes** и **срок действия**.

ДЕМО Демо: полный цикл «задача → MR → main»

```
1. Issue #1 «Добавить эндпоинт /version в README»   → метка docs, исполнитель — я
2. В issue: Create merge request → ветка 1-dobavit-endpoint
```

```
git fetch origin
git switch 1-dobavit-endpoint
echo -e "\n## API\n- GET /version — версия приложения" >> README.md
git commit -am "docs: описан /version (#1)"
git push
```

```
3. MR: Ready → Reviewer → комментарий → suggestion → Apply
4. Merge (squash, delete source branch)  → issue #1 закрыт автоматически
5. Repository → Graph: история
```

Пайплайн? Пока нет

Добавим простейший `.gitlab-ci.yml`:

```
hello:
  script:
    - echo "Привет из GitLab CI!"
```

Пайплайн появится, но job будет висеть в **pending**:

> *This job is stuck because the project doesn't have any runners online assigned to it.*

GitLab **сам не выполняет** jobs — нужен **Runner**. Это следующая тема.

Итоги

- **Группа → проект**; роли наследуются; видимость Private/Internal/Public
- Доступ к Git: **HTTP + PAT** или **SSH-ключ**
- Работа: **issue → ветка → MR → review → merge**, `main` защищена
- Container Registry проекта и токены: PAT, deploy token, CI job token
- Без runner'а пайплайны не выполняются

Практика 08

Свой проект notes-api в GitLab

Задание — на платформе testlms.kz